

Damping Sweep & Analysis Campaign Manual

compass_sim — scripts *damping_sweep.py*, *damping_sweepV02.py*,
sweep_damping_hysteresis.py, *damping_sweep_analysis.py*

Manual revision: July 2026

J. P. Sinnecker

July 9, 2026

Contents

1	Overview	2
2	Physical Model and Parameter Derivation	2
2.1	Equations of motion	2
2.2	Quality factor Q	2
2.3	Needle geometry and physical parameters	3
3	damping_sweep.py — Serial Campaign	3
3.1	Operation	3
3.2	Command-line interface	3
3.3	Output structure	4
4	damping_sweepV02.py — Parallel Campaign	4
4.1	Parallel execution with worker sandboxing	4
4.2	Incremental manifest and resume	4
4.3	V02-specific flags	5
5	sweep_damping_hysteresis.py — Standalone Script	5
5.1	Corrections applied (July 2026)	5
5.2	Limitations	5
6	damping_sweep_analysis.py — Post-Processing	5
6.1	Hysteresis loop metrics	5
6.2	Avalanche detection — two methods	6
6.3	Power-law exponent τ — MLE fit (Clauset–Shalizi–Newman)	6
6.4	Power-law versus log-normal likelihood-ratio test (LRT)	6
6.5	Waiting-time distribution between avalanches	7
6.6	Temporal autocorrelation of $M(t)$	7
6.7	Output files	7
6.8	Analysis command-line interface	8
7	Practical Recipes	8
7.1	Pipeline verification (quick test)	8
7.2	Standard campaign (CPU, parallel)	8
7.3	Resume an interrupted campaign	8
7.4	LAW3M 2026 — Recommended production campaign	8
7.5	Demo animations (underdamped vs. overdamped)	9

8	Physical Parameter Reference	9
9	Notes and Caveats	10

1 Overview

This manual describes the Python simulation-campaign suite built to study the central scientific question of the `compass_sim` project:

“Does the viscous damping coefficient control the universality class of avalanches and the shape of the hysteresis loop in classical two-dimensional dipolar arrays, and does this depend on the lattice geometry (square, triangular, honeycomb)?”

The suite consists of four scripts:

Script	Role
<code>damping_sweep.py</code>	Sequential campaign runner (hysteresis + relaxation)
<code>damping_sweepV02.py</code>	Parallel campaign runner (<code>ProcessPoolExecutor</code>); same physics as V01, adds <code>-n_workers</code> and <code>-resume</code>
<code>sweep_damping_hysteresis.py</code>	Earlier standalone script; now physically consistent with V01/V02 after corrections
<code>damping_sweep_analysis.py</code>	Post-processing: metrics, power-law fitting, waiting-time distribution, autocorrelation, plots

The sweep execution proceeds in two stages per configuration:

Stage 1 — Hysteresis. A five-ramp cycle ($0 \rightarrow +B_{\max} \rightarrow 0 \rightarrow -B_{\max} \rightarrow 0 \rightarrow +B_{\max}$) sweeps the external field to trace the hysteresis loop and capture avalanche events.

Stage 2 — Free relaxation. The external field is set to zero and the system relaxes freely from its post-hysteresis state; magnetic-domain sizes and counts are measured in the final configuration.

2 Physical Model and Parameter Derivation

2.1 Equations of motion

Each needle i satisfies Newton’s second law for rotation:

$$I \ddot{\theta}_i = \tau_i^{\text{dip}} + \tau_i^{\text{ext}} - b \dot{\theta}_i \quad (1)$$

where I is the moment of inertia [kg m^2], b is the viscous damping coefficient [N m s/rad], and the torques arise from dipolar coupling and the external field.

2.2 Quality factor Q

The dimensionless quality factor characterises the dynamical regime:

$$Q = \frac{\omega_0 I}{b}, \quad \omega_0 = \sqrt{\frac{m B_{\text{ref}}}{I}}, \quad B_{\text{ref}} = \frac{\mu_0}{4\pi} \frac{2m}{r_{\text{nn}}^3} \quad (2)$$

where m is the needle magnetic moment [A m^2], r_{nn} is the nearest-neighbour distance ($= 2R$ for all implemented geometries), and B_{ref} is the dipolar field between nearest neighbours.

Three dynamical regimes are identified:

Regime	Condition	Physical behaviour
Underdamped	$Q \gg 1$	Needles oscillate before settling; chain-reaction avalanches
Critically damped	$Q \approx 1$	Fastest settling with no overshoot
Overdamped	$Q \ll 1$	Slow monotonic relaxation; narrow hysteresis loops

2.3 Needle geometry and physical parameters

All scripts derive moment of inertia I and magnetic moment m from the physical geometry of the needle blade and its pivot pin using `compass.py`'s authoritative functions:

`compute_inertia_from_geometry(needle_len, needle_width, thickness, density, pivot_*)` (3)

`compute_moment_from_geometry(needle_len, needle_width, thickness, Ms)` (4)

where the needle geometry is derived from the circumscribed radius R and the needle fraction f :

$$L_{\text{needle}} = f \times 2R, \quad W_{\text{needle}} = 0.22 \times L_{\text{needle}}, \quad f = 0.80 \text{ (default)} \quad (5)$$

The pivot pin contributes an additive term to I (Eq. 3); if `-pivot_radius = 0` (the default), the pivot contribution is zero.

Important: in earlier versions of `sweep_damping_hysteresis.py`, the module-level fallback constants `cs.MOMENT_DEFAULT` and `cs.INERTIA_DEFAULT` were used instead of the geometry-derived values. This is now fixed — all scripts call the same geometry functions as `compass.py main()`.

3 damping_sweep.py — Serial Campaign

3.1 Operation

`damping_sweep.py` imports `compass.py` and calls its `relax()` function for each combination:

$$\text{geometry} \times \text{damping}/Q \times \text{seed}$$

Runs are executed sequentially on a single CPU core.

The damping grid is built once using the *square* geometry as reference and then the same physical damping values b are applied to the other geometries. Because r_{m} (and therefore Q) differs slightly between geometries, the actual Q of every run is computed and stored in its metadata JSON.

3.2 Command-line interface

Flag	Default	Description
<code>-out_dir</code>	<i>required</i>	Output directory
<code>-grid_n</code>	30	Grid dimension $N = M$ (total needles = N^2)
<code>-n_seeds</code>	5	Random seeds per (geometry, damping)
<code>-n_dampings</code>	8	Points in the Q grid
<code>-Q_min</code>	0.05	Minimum quality factor
<code>-Q_max</code>	15.0	Maximum quality factor
<code>-geometries</code>	<i>all three</i>	Comma-separated: <code>square,triangular,honeycomb</code>
<code>-R</code>	0.025	Circumscribed needle radius [m]
<code>-B_max_factor</code>	8.0	$B_{\text{max}} = \text{factor} \times B_{\text{ref}}$
<code>-t_sim_periods</code>	40.0	Simulation duration in units of $T_0 = 2\pi/\omega_0$
<code>-pbc</code>	0	Periodic boundary conditions (0 = off)
<code>-use_gpu</code>	0	GPU acceleration via CuPy (0 = off)
<code>-skip_relax_stage</code>	0	Omit Stage 2 (domain statistics)
<code>-damping_noise</code>	0.0	Relative per-needle damping variation (e.g. 0.05 = 5%)

Flag	Default	Description
-needle_frac	0.80	Blade length fraction of $2R$
-needle_thickness	0.0004	Steel blade thickness [m]
-steel_density	7850.0	Blade steel density [kg m^{-3}]
-steel_Bsat	<i>None</i>	Override saturation flux density [T]
-moment	<i>None</i>	Override magnetic moment [A m^2]
-inertia	<i>None</i>	Override moment of inertia [kg m^2]
-pivot_radius	0.0	Pivot pin radius [m]
-pivot_thickness	0.0	Pivot pin thickness [m]
-pivot_density	8500.0	Pivot pin material density [kg m^{-3}]
-pivot_mass	<i>None</i>	Override pivot mass directly [kg]
-dt_factor	0.05	Time step Δt as fraction of T_0
-noise	1.5	Initial angle noise [rad]
-domain_tol	15.0	Angular tolerance for domain clustering [$^\circ$]
-quick_test	0	If 1: $N = 12$, 1 seed, 3 dampings, 8 periods — pipeline check

3.3 Output structure

```

<out_dir>/
+-- manifest.csv           # one row per run (stage, tag, Q, paths, ...)
+-- hysteresis/
|   +-- square_damp00_seed00.csv  # columns: t, B, M, S
|   +-- ...
+-- relax/
|   +-- square_damp00_seed00.csv
|   +-- ...
+-- meta/
    +-- square_damp00_seed00_hyst.json  # full physical parameters + Q
    +-- square_damp00_seed00_relax.json # + domain_sizes list

```

CSV column names are *t*, *B*, *M*, *S* where *S* is the integer spin-flip count at each integration step (number of needles that reversed orientation).

4 damping_sweepV02.py — Parallel Campaign

V02 retains the full parameter set of V01 and adds two features:

4.1 Parallel execution with worker sandboxing

`ProcessPoolExecutor` distributes runs across `-n_workers` processes. Each worker is initialised in a private scratch directory (`_scratch/worker_xxxx/`) so that `hysteresis_loop.csv` and other simulator side-effect files do not collide.

Never combine `-n_workers > 1` with `-use_gpu 1`: multiple processes cannot safely share a single GPU context.

4.2 Incremental manifest and resume

`manifest.csv` is appended and flushed after *each* completed run. An interrupted campaign can be resumed with `-resume 1`; the script reads the existing manifest, identifies completed (stage, tag) pairs, and skips them.

4.3 V02-specific flags

Flag	Default	Description
<code>-n_workers</code>	1	Number of parallel processes
<code>-resume</code>	0	If 1, skips runs already in <code>manifest.csv</code>

5 sweep_damping_hysteresis.py — Standalone Script

This is an earlier standalone script that runs only Stage 1 (hysteresis) with hard-coded geometry and a per-run scratch directory. Its output is compatible with `damping_sweep_analysis.py`.

5.1 Corrections applied (July 2026)

Three bugs present in the original version were corrected:

1. **CUTOFF fixed:** the interaction cutoff was 8.0 (treated as metres by `relax()`, which uses units of r_{nn} , giving a physically incorrect $8 r_{nn}$ cutoff instead of the intended $3.5 r_{nn}$). Corrected to `CUTOFF = 3.5`, consistent with V01/V02 and the `compass.py` CLI default.
2. **MOMENT/INERTIA fixed:** the script used `cs.MOMENT_DEFAULT` and `cs.INERTIA_DEFAULT` (module-level fallback values) instead of calling the geometry derivation functions. Now uses `cs.compute_moment_from_geometry()` and `cs.compute_inertia_from_geometry()`, making results cross-comparable with V01/V02.
3. **$B_{\max}$ and CSV columns fixed:** `B_MAX` was a hard-coded constant; it is now computed dynamically as `B_MAX_FACTOR × B_ref`. CSV column names corrected from `B_proj`, `M_proj` to `B`, `M` so that `damping_sweep_analysis.py` can read the files without modification. A `damp_idx` field was also added to metadata.

5.2 Limitations

`sweep_damping_hysteresis.py` runs sequentially (no `-n_workers`) and does not generate Stage 2 relaxation data. For new campaigns, prefer `damping_sweepV02.py`.

6 damping_sweep_analysis.py — Post-Processing

The analysis script reads the directory produced by any of the sweep scripts (identified via `manifest.csv`) and computes all metrics.

6.1 Hysteresis loop metrics

Loop area. $A = \frac{1}{2} \left| \sum_k B_k M_{k+1} - B_{k+1} M_k \right|$ (shoelace formula applied to the closed loop in the B – M plane). Measures energy dissipated per cycle.

Coercive field B_c . Linear interpolation of B where M crosses zero. The median over all such crossings in the cycle is reported.

Remanence M_r . Linear interpolation of M at the time-indices where $B = 0$. The median is reported.

6.2 Avalanche detection — two methods

Both methods are applied to every hysteresis run; their results are saved separately so that the two approaches can be compared.

Method 1: dM/dB proxy (MAD-based). The field cycle is split into monotonic ramps at its turning points. Within each ramp, the derivative dM/dB is computed and a robust threshold is set using the Median Absolute Deviation:

$$\text{threshold} = \text{median}(dM/dB) + \kappa \cdot \text{MAD}(dM/dB) \quad (6)$$

where $\kappa = \text{-mad_threshold}$ (default 4.0). Contiguous steps above the threshold form one avalanche; its size is $s = \sum |dM|$ over those steps.

Method 2: spin-flip counter (S column). A cleaner alternative using the integer **S** column written by `compass.py` at each integration step. An avalanche is a maximal run of consecutive steps with $S > 0$; its size is $\sum S$ over the run. This method involves no numerical differentiation and is immune to numerical drift near inflection points.

The S-column method also returns the **inter-arrival times** (waiting times between consecutive avalanche starts), used in Section 6.5.

6.3 Power-law exponent τ — MLE fit (Clauset–Shalizi–Newman)

To stabilise the MLE estimate, avalanche sizes are *aggregated across all seeds* for each (geometry, damp_idx) combination before fitting. The procedure follows Clauset, Shalizi & Newman 2009:

1. For candidate lower bounds $x_{\min}$ (quantiles of the sample), compute the MLE estimator for the continuous power-law exponent:

$$\hat{\tau} = 1 + n \left[\sum_{i=1}^n \ln \left(\frac{s_i}{x_{\min}} \right) \right]^{-1} \quad (7)$$

2. Evaluate the KS distance D between the empirical CDF and the theoretical power-law CDF.
3. Select the $x_{\min}$ that minimises D .
4. Discard fits with fewer than 15 tail points.

If the `powerlaw` package is installed it is used instead of the built-in fallback (same methodology; the package is the canonical Clauset 2009 reference implementation).

6.4 Power-law versus log-normal likelihood-ratio test (LRT)

For each aggregated (geometry, damp_idx) sample, a Vuong-style likelihood-ratio test distinguishes whether the data favours a power law over a log-normal, following Clauset 2009 §4:

$$R = \sum_i \ln \frac{p_{\text{PL}}(s_i)}{p_{\text{LN}}(s_i)}, \quad p = \text{erfc} \left(\frac{|R|}{\sqrt{2} \hat{\sigma}_R \sqrt{n}} \right)$$

$R > 0$ and $p < 0.10$ is recorded as “power-law preferred”. The columns `lrt_R`, `lrt_p`, and `lrt_favors_powerlaw` appear in `powerlaw_fits.csv` and `powerlaw_fits_S.csv`.

6.5 Waiting-time distribution between avalanches

From the S-column inter-arrival times, the analysis fits two models:

Exponential (Poisson process). MLE: $\hat{\lambda} = 1/\langle\tau\rangle$.

Power law. MLE as above.

The reported metrics include:

- Mean inter-arrival time $\langle\tau_w\rangle$.
- Coefficient of variation $CV = \sigma_{\tau_w}/\langle\tau_w\rangle$ ($CV \gg 1$ signals bursty / SOC-like timing).
- LRT statistic: $\Lambda = 2(\mathcal{L}_{PL} - \mathcal{L}_{Exp})$; positive values indicate departure from Poisson.

6.6 Temporal autocorrelation of $M(t)$

Motivated by the 2025 echo-state / memory-capacity result for ASI (*Taniguchi, Sci. Rep. 2025*), the script computes the normalised autocorrelation function

$$C(\ell) = \frac{\langle M_c(t) M_c(t + \ell) \rangle}{\text{var}(M)} \quad (8)$$

for lags up to 25% of the run length. The correlation time is

$$\tau_{\text{corr}} = \int_0^{\ell^*} C(\ell) d\ell \quad (9)$$

where ℓ^* is the lag of the first zero-crossing of C . The prediction is that τ_{corr} peaks near $Q \approx 1$ (critical damping).

6.7 Output files

File	Contents
summary_hysteresis.csv	Per-run metrics: area, B_c , M_r , avalanche counts (both methods), waiting-time statistics, τ_{corr}
summary_relax.csv	Per-run domain statistics from Stage 2
aggregated_loop_metrics.csv	Mean $\pm$ std of area, B_c , M_r aggregated over seeds
powerlaw_fits.csv	MLE results (τ , τ_{err} , x_{min} , KS, LRT) using the dM/dB avalanche sizes
powerlaw_fits_S.csv	Same, using the spin-flip counter (S-column) sizes
area_vs_Q.png	Loop area vs. Q (by geometry)
Bc_vs_Q.png	Coercive field vs. Q
Mr_vs_Q.png	Remanence vs. Q
tau_vs_Q.png	Central result: avalanche exponent τ vs. Q (dM/dB method)
tau_vs_Q_Scol.png	Same, using S-column sizes (cross-check)
avalanche_histograms.png	Log-log $P(s)$ histograms with MLE fit lines
domain_size_vs_Q.png	Mean domain size and domain count vs. Q
waiting_time_vs_Q.png	Mean IAT and LRT statistic vs. Q
autocorr_vs_Q.png	Correlation time τ_{corr} of $M(t)$ vs. Q

6.8 Analysis command-line interface

Flag	Default	Description
-sweep_dir	<i>required</i>	Directory written by a sweep script
-out_dir	<i>required</i>	Where to save analysis results
-mad_threshold	4.0	MAD multiplier κ for Method 1 avalanche detection
-forc_dir	<i>None</i>	(Stub) Directory for FORC output; not yet implemented

7 Practical Recipes

7.1 Pipeline verification (quick test)

```
# 12x12 grid, 1 seed, 3 dampings, 8 periods -- runs in seconds
python3 damping_sweep.py --out_dir "$(pwd)/quicktest" --quick_test 1

# Analyse immediately
python3 damping_sweep_analysis.py \
    --sweep_dir "$(pwd)/quicktest" \
    --out_dir "$(pwd)/quicktest/analysis"
```

7.2 Standard campaign (CPU, parallel)

```
# 8 dampings x 3 geometries x 5 seeds = 240 hysteresis runs + 240 relax
python3 damping_sweepV02.py \
    --out_dir "$(pwd)/results" \
    --n_workers 4

# Post-process
python3 damping_sweep_analysis.py \
    --sweep_dir "$(pwd)/results" \
    --out_dir "$(pwd)/results/analysis"
```

7.3 Resume an interrupted campaign

```
python3 damping_sweepV02.py \
    --out_dir "$(pwd)/results" \
    --n_workers 4 --resume 1
```

7.4 LAW3M 2026 — Recommended production campaign

This is the parameter set designed to produce the key figures for the LAW3M 2026 conference contribution on “Inertial effects on avalanche universality classes in classical dipolar lattices”.

```
# STAGE 1: Run the campaign overnight (approx. 6-8 h on 4 cores)
# 14 Q-values x 3 geometries x 8 seeds = 336 hysteresis + 336 relax runs

python3 damping_sweepV02.py \
    --out_dir ~/results/law3m_2026 \
    --grid_n 30 \ # 30x30 = 900 needles per run
    --n_seeds 8 \ # 8 noise realisations per (geometry,Q)
    --n_dampings 14 \ # 14 Q-points from Q=0.02 to Q=30
    --Q_min 0.02 \ # deep overdamped: monotonic M(B)
```

```

--Q_max      30.0      \  # deep underdamped: inertial overshoot
--geometries square,triangular,honeycomb \
--t_sim_periods 80.0    \  # 80 T0 per cycle (5 segments)
--B_max_factor 8.0      \  # B_max = 8 x B_ref (ensures saturation)
--damping_noise 0.05    \  # 5% per-needle variation (realistic disorder)
--dt_factor 0.04        \  # finer step; important at large Q
--n_workers 4           \
--resume 1

# STAGE 2: Analysis
python3 damping_sweep_analysis.py \
  --sweep_dir ~/results/law3m_2026 \
  --out_dir   ~/results/law3m_2026/analysis \
  --mad_threshold 3.5

```

Key figures for the conference presentation:

1. `tau_vs_Q.png` and `tau_vs_Q_Scol.png` — the central physics result: does τ vary with Q ? Does it differ by geometry?
2. `autocorr_vs_Q.png` — memory/correlation time vs. Q ; tests the echo-state prediction.
3. `avalanche_histograms.png` — visual $P(s)$ comparison between the underdamped ($Q \approx 30$) and overdamped ($Q \approx 0.02$) limits.
4. Side-by-side video frames from `compass.py` direct calls (see below) illustrating chain-reaction vs. sluggish switching.

7.5 Demo animations (underdamped vs. overdamped)

```

# Underdamped (Q~15): inertial overshoot, chain-reaction avalanches
python3 compass.py --N 20 --M 20 --geometry square \
  --field_mode hysteresis --B_ext 0.003 --t_sim 25 \
  --damping 2.4e-7 --dt_factor 0.04 \
  --frame_dir ~/frames/underdamped --frame_every 5

# Overdamped (Q~0.05): sluggish monotonic switching
python3 compass.py --N 20 --M 20 --geometry square \
  --field_mode hysteresis --B_ext 0.003 --t_sim 25 \
  --damping 7.0e-5 --dt_factor 0.04 \
  --frame_dir ~/frames/overdamped --frame_every 5

```

8 Physical Parameter Reference

Default values for a tabletop steel needle with $R = 0.025$ m and $f = 0.80$:

Quantity	Symbol	Value	Unit
Needle half-length	$L/2$	20.0	mm
Needle width	W	4.4	mm
Needle thickness	e	0.4	mm
Steel density	ρ	7850	kg m^{-3}
Saturation magnetisation	M_s	1.59×10^6	A m^{-1}
Magnetic moment (derived)	m	$\approx 5.6 \times 10^{-5}$	A m^2
Moment of inertia (derived)	I	$\approx 7.6 \times 10^{-9}$	kg m^2
Nearest-neighbour distance	r_{nn}	50	mm
Dipolar reference field	B_{ref}	≈ 0.36	mT
Natural frequency	ω_0	≈ 22.8	rad s^{-1}
Natural period	T_0	≈ 0.276	s
Default damping	b	5×10^{-8}	N m s rad^{-1}
Corresponding Q		≈ 3.5	

9 Notes and Caveats

- **Avalanche statistics require large samples.** A single 30×30 hysteresis run may yield only $\mathcal{O}(10)$ – $\mathcal{O}(100)$ events. The analysis aggregates over all seeds before the MLE fit; fewer than 8 seeds at any (geometry, Q) point typically gives unstable τ estimates.
- **Method 2 (S-column) preferred.** The dM/dB MAD method is sensitive to the threshold multiplier κ and to numerical noise near field inflection points. The S-column method counts discrete needle reversals with no free parameters; use `tau_vs_Q_Scol.png` as the primary result and `tau_vs_Q.png` as a cross-check.
- **Do not mix GPU with $n_{\text{workers}} > 1$.** Multiple worker processes cannot safely share a CUDA context.
- **Pivot parameters default to zero.** If the physical experiment uses a metallic pivot pin that contributes to I , specify `-pivot_radius`, `-pivot_thickness`, and `-pivot_density` (or `-pivot_mass`) to maintain consistency between the simulation and the tabletop apparatus.
- **The cutoff is in units of r_{nn} .** The `-cutoff` argument passed to `relax()` is expressed as a multiple of the nearest-neighbour distance ($r_{\text{nn}} = 2R$), *not* in metres. The default value of 3.5 includes up to $\approx$ third-shell neighbours.